

Arquitetura Santa Cecília

Sistemas escaláveis, seguros e preparados para o mundo real

8 de outubro de 2026 · Build Arch Desktop · Demonstração de 25 minutos

Repositório público

<https://github.com/DevKaue/arquitetura-santa-cecilia>

Referência: main. Importação por URL dentro do roteiro principal.

Como usar este guia

1. Resumo executivo: alinhar as decisões e os slides com a chefia.
2. Roteiro: acompanhar ações, falas, parâmetros e transições.
3. Cartão do apresentador: consultar a sequência rapidamente.
4. Perguntas e respostas: conduzir a discussão por tema.

Arquitetura final: v9 · 19 blocos · 25 relações · 3 laboratórios.

Resumo executivo para revisão

Evento: palestra de arquitetura de software, Santa Cecília, 8 de outubro de 2026. **Demonstração:** 25 minutos no Build Arch Desktop; formato reduzido de 20 minutos disponível. **Repositório:** [DevKaue/arquitetura-santa-cecilia](#). **Projeto principal:** Arquitetura Santa Cecília, revisão v9.

1. Objetivo da demonstração

Transformar os conceitos dos slides em decisões visíveis: distribuir leituras, proteger a compra, retirar trabalho posterior do tempo de resposta e identificar o gargalo antes de aumentar a infraestrutura. A plateia acompanha três jornadas: consulta ao catálogo, compra transacional e processamento após a compra.

O exemplo mantém o caso de catálogo e compras apresentado no material da palestra. O nome do projeto é **Arquitetura Santa Cecília**. O desenho final reúne 19 blocos e 25 relações lógicas; oito versões anteriores mostram a evolução. Três laboratórios isolam a capacidade da API e do banco.

2. Problema e premissas

Premissa	Valor do exercício	Uso na explicação
Tráfego normal	2.000 requisições/min	Referência para comparação
Pico de campanha	20.000 requisições/min	Aumento de 10 vezes
Perfil do negócio	95% leitura; 5% escrita	Separar catálogo e compra
Cache HIT ilustrativo	90% das leituras	Calcular alívio no banco, sem confundir com roteamento do simulador

Com essas premissas, o pico produz 19.000 leituras e 1.000 escritas por minuto. Um HIT de 90% deixa 1.900 leituras chegando ao banco. Essa conta explica uma hipótese de arquitetura; a simulação de carga não aplica automaticamente a proporção 95/5 nem o cache HIT.

3. Decisões centrais

Tema	Decisão	Consequência a discutir
Leitura	CDN, cache-aside e réplicas	Ganho de latência/capacidade com atraso tolerado no catálogo
Compra	Autorizar o dono; reservar no writer	Evitar venda sem estoque e acesso indevido a pedidos
Pagamento	Idempotência e reconciliação	Timeout mantém resultado desconhecido até confirmação
Assíncrono	Outbox, relay, consumidor e DLQ	Entrega pode repetir; o efeito precisa ser controlado
Operação	Métricas, canary e rollback	Detectar degradação e reduzir o impacto de mudança
Recuperação	Writer único e região secundária	RPO/RTO propostos exigem teste de restauração e failover

4. Evidência que será mostrada

No cenário temporal de pico 4x, com a mesma entrada, os Labs 01 e 02 produzem latência estimada de 895 ms e erro estimado de 68%. A API cresce de uma para seis instâncias, mas o banco permanece limitante. No Lab 03, a capacidade cadastrada do banco aumenta e os valores passam a 48 ms e 0,05%.

Na carga ao vivo 10x, são geradas 10.000 requisições em 30 segundos virtuais. Após o limite de drenagem, Labs 01/02 deixam aproximadamente 2.650 pendentes; o Lab 03 termina sem pendências. São resultados do modelo do Build Arch, sem representar benchmark HTTP ou SLA real.

O Compose declara nove serviços. A inferência estática pelo motor do Build Arch identificou 23 blocos e 22 relações, incluindo redes, volume, imagens e abstrações de software. Esse resultado permite explicar a revisão humana de um mapa extraído do GitHub.

5. Alinhamento com os slides

Slides	Ponto de revisão
3-4	Retomar o caso e explicitar que os números são premissas
11	Consistência depende das operações e do isolamento adotado
14	TLS pode terminar na entrada; nova sessão protege o backend
18	Reserva de estoque ocorre no checkout; notificação pode esperar
19-21	Relacionar segurança, observabilidade, canary e limite de escala
22-23	Diferenciar simulação temporal 4x, carga ao vivo 10x e gargalo compartilhado

6. Escopo e revisão proposta

O repositório publica o modelo, os manifests e as notas técnicas. Os servidores são um esqueleto para inferência: catálogo sintético, health endpoints e checkout com resposta 501. Pagamento, consumidor e relay reais não estão implementados. A sintaxe do Compose foi validada; os containers não foram executados.

Para revisar, abrir o backup principal, acompanhar o roteiro e conferir: separação leitura/compra, reserva concorrente, idempotência, evento de confirmação de compra, limites de retry, DLQ, redundância e metas de recuperação. Revisar especialmente as correções propostas para os slides 14 e 18 antes do ensaio conjunto.

Roteiro da demonstração no Build Arch

Duração: 25 minutos. **Formato:** demonstração guiada, com importação por URL do GitHub no roteiro principal.

Resultado esperado: a plateia explica por que a arquitetura evolui e por que escalar a API pode manter o mesmo gargalo.

Preparação do apresentador

1. Abrir **Arquitetura Santa Cecília** no Build Arch Desktop, em **Trabalho atual · v9**. Conferir **Salvo agora** e o enquadramento do canvas.
2. Conferir os projetos **Santa Cecília | Lab 01 | Inicial: banco limita**, **Santa Cecília | Lab 02 | Escalar a API não resolve** e **Santa Cecília | Lab 03 | Capacidade no gargalo**.
3. Deixar disponível a URL **<https://github.com/DevKaue/arquitetura-santa-cecilia>** e a referência **main**.
4. Ensaiar uma importação pelo painel **Analisar projeto**, usando **GitHub público**. A demonstração usa essa URL; backups ficam como recuperação do material.
5. Abrir o cartão do apresentador. Não alterar blocos durante uma execução de carga: a edição limpa a execução.

Visão do tempo

Minutos	Etapas	Mensagem
0-2	Problema	Leitura e compra têm exigências diferentes
2-6	Evolução do desenho	Cada componente responde a uma necessidade
6-9	Compra e pós-compra	Transação local e integração externa exigem controles distintos
9-15	Três laboratórios	Mais APIs não corrigem um banco limitante
15-18	Pico 10x	Pendência mostra acúmulo de trabalho
18-22	Importar do GitHub	Código produz evidências; arquitetura exige revisão
22-25	Falha, decisão e exportação	Medir, conter impacto e registrar a decisão

0:00-2:00 - Abrir o problema

Ação no Desktop: mostrar o projeto completo em 2D e apontar usuários, catálogo e checkout. Relacionar com os slides 3-4.

Fala sugerida: "A maioria das pessoas consulta produtos. Uma parte menor compra, mas uma compra sem estoque ou uma cobrança duplicada custa muito mais caro. Vamos acompanhar o que pode aceitar atraso e o que exige uma decisão confiável."

Mostrar: 2.000 req/min normal, 20.000 no pico e perfil 95/5. Explicar que são premissas do exercício. Perguntar: "Qual dependência vocês investigariam primeiro durante o pico?"

Transição: "A arquitetura cresce quando aparece uma necessidade. Vamos ver a sequência."

2:00-6:00 - Percorrer a evolução

Ação no Desktop: usar **Versão da arquitetura** para mostrar brevemente as etapas anteriores; retornar a **Trabalho atual · v9**. Relacionar com os slides 13-19.

Apontar	Explicação principal
DNS e CDN/WAF	DNS resolve o destino; CDN aproxima conteúdo público; WAF reduz ataques na borda
Load Balancer e APIs	Entrada distribui chamadas; instâncias stateless podem crescer
Redis e réplicas A/B	HIT responde pelo cache; MISS escolhe uma réplica; leitura pode atrasar
Principal + standby	Existe um writer; standby protege contra falha, sem duplicar escrita
Outbox, relay, fila e worker	Trabalho posterior à compra segue por entrega assíncrona
Observabilidade, CI/CD e DR	Métricas orientam operação; entrega gradual contém regressão; recuperação precisa de ensaio

Fala sugerida: “As caixas mostram responsabilidades. Começar com um monólito modular pode ser adequado; separar implantação passa a fazer sentido quando volume, equipe e isolamento de falhas justificam o custo.”

Atenção: DNS, replicação, telemetria e deploy são relações lógicas. O mapa completo não deve ser usado como uma cadeia de processamento de cada requisição. Custo e disponibilidade do painel usam hipóteses.

6:00-9:00 - Explicar a compra correta

Ação no Desktop: apontar **06 API Checkout**, **10 Principal + standby**, **18 Publicador da outbox**, **11 Eventos pós-compra**, **12 Worker notificações** e **19 Mensagens para análise**. Abrir **Modelagem de dados** no banco principal.

Sequência para explicar:

1. Validar identidade e autorizar o dono do pedido. Conferir chave de idempotência e hash da solicitação.
2. Reservar estoque com atualização condicional; gravar pedido, itens e evento de outbox na mesma transação.
3. Fazer commit. Chamar pagamento externo com idempotência, fora da transação SQL.
4. Reconciliar a confirmação. Em nova transação, confirmar a compra e gravar o evento [purchase_confirmed](#).
5. O relay publica após commit, aguarda confirmação do broker e marca a publicação. Uma queda entre publicação e marcação pode gerar duplicata.
6. O consumidor controla o efeito pelo evento e confirma a mensagem após o resultado durável. Tentativas esgotadas vão para a DLQ, com análise e reprocessamento controlado.

Mostrar no schema: [orders.owner_subject](#), [idempotency_key](#), [request_hash](#); [reservations.expires_at](#); [outbox](#); chave composta de [processed_events](#).

Fala sugerida: “A tela pode exibir estoque atrasado. A decisão de reservar precisa vencer a concorrência no writer. A outbox mantém o pedido e a intenção de publicar juntos; a entrega ainda pode repetir.”

9:00-15:00 - Comparar os três laboratórios

Configuração comum: **Simular arquitetura** > carga inicial **2.000 req/min** > perfil **Pico** > **4,0x** > duração **10 min** > **Nenhuma falha** > **Executar simulação**. A duração é virtual.

Projeto	Ação	O que interpretar
Lab 01	Executar e apontar o primeiro gargalo	Banco transacional limita o fluxo
Lab 02	Repetir sem mudar os parâmetros	API cresce de 1 para 6 instâncias; banco continua igual
Lab 03	Repetir novamente	Capacidade cadastrada do banco aumenta; o limitador recebe a mudança

Resultado do mesmo pico	Lab 01	Lab 02	Lab 03
Instâncias de API	1	6	6
Capacidade do banco, req/min	3.000	3.000	20.000
Latência estimada, ms	895	895	48
Erro estimado	68%	68%	0,05%

Fala sugerida: “Mais cópias da API continuam chegando ao mesmo banco. Primeiro medimos o limitador; depois decidimos entre otimizar queries e índices, controlar concorrência ou aumentar a capacidade necessária.”

Limite a declarar uma vez: latência e erro são heurísticas do Build Arch. O Lab 03 representa uma capacidade informada; não prova que adicionar réplica de leitura acelera escrita.

15:00-18:00 - Mostrar a campanha 10x

Ação no Desktop: no Lab 01, abrir **Teste de carga ao vivo**. Usar entrada **Usuários, 200 usuários, 10 ações/min, 30 segundos**, sem falha. Iniciar e mostrar a base de 2.000 req/min.

Limpar a execução. Mudar apenas para **2.000 usuários**, mantendo **10 ações/min** e **30 s**: são 20.000 req/min. Mostrar a fila no banco. No Lab 03, repetir o pico com os mesmos valores.

Resultado: 10.000 requisições são geradas. Labs 01/02 deixam aproximadamente 2.650 pendentes após a drenagem máxima; Lab 03 conclui sem pendências em 33 segundos virtuais. Pendência é acúmulo de trabalho, não comprovação de perda de pedido.

Fala sugerida: “A fila mostra que a entrada está maior que a capacidade de processamento. Escalar precisa respeitar o banco e o orçamento.”

Limite do motor: o temporal permite até 4x; a carga ao vivo demonstra 10x pelos usuários. Os motores têm cálculos diferentes. Ramificações da carga ao vivo dividem tráfego igualmente; por isso usamos os Labs lineares.

18:00-22:00 - Importar a arquitetura do GitHub

URL: <https://github.com/DevKaue/arquitetura-santa-cecilia> **Referência:** main

1. Abrir **Analisar projeto** no Desktop e selecionar **GitHub público**.
2. Colar a URL raiz do repositório. Informar **main** no campo de referência, se exibido.
3. Executar a análise. Mostrar as evidências de [docker-compose.yml](#), redes, volume e Dockerfiles.
4. Apontar o relay dependendo de PostgreSQL e RabbitMQ, e o checkout dependendo do PostgreSQL.
5. Abrir a arquitetura gerada como outro projeto. Comparar com o desenho revisado e retornar à **Arquitetura Santa Cecília**.

Fala sugerida: “O repositório revela serviços e dependências declaradas. Precisamos conferir se cada relação é inicialização, comunicação ou uma hipótese. O código ajuda a começar; a decisão de arquitetura vem da revisão.”

Resultado verificado pelo motor: nove serviços do Compose geraram 23 blocos e 22 relações. O mapa inclui redes, imagens, volume e abstrações de software; não representa 23 microserviços. `depends_on` não prova uma chamada de negócio nem garante prontidão sem uma condição adequada.

Se a rede falhar: usar os backups para continuar a explicação e mostrar as evidências registradas em [docs/validacao/inferencia-verificada.json](#). Deixar claro que a importação por URL não foi realizada naquele momento; retomar o teste após restabelecer a rede.

22:00-25:00 - Falha, decisão e registro

Ação no Desktop: no Lab 03, injetar falha do **Banco transacional**. Explicar que o controle derruba o bloco inteiro; não testa failover de uma réplica PostgreSQL. Retornar ao mapa completo.

Discutir brevemente pagamento lento, retry limitado, DLQ e recuperação. O catálogo pode continuar quando uma integração de pagamento está indisponível. Timeout externo mantém resultado desconhecido até consulta ou reconciliação. $RPO \leq 5 \text{ min}$ e $RTO \leq 30 \text{ min}$ são metas propostas, que exigem ensaio.

Fechamento sugerido: “Cada decisão resolve um problema e traz um custo. O próximo passo é medir o sistema real, testar concorrência e falhas e confirmar se as premissas atendem ao negócio.”

Mostrar **Arquivo > Salvar backup**. Uma nova decisão merece revisão e exportação; os arquivos publicados em [buildarch/](#) permitem abrir o material em outra máquina.

Adaptação para 20 ou 30 minutos

20 min: reduzir evolução do mapa para 3 min; comparar configuração do Lab 02 sem repetir a execução inteira; explicar apenas quatro tabelas na compra. Manter a importação do GitHub e os dois cenários de carga principais.

30 min: manter o roteiro de 25 min e reservar 5 min para perguntas sobre idempotência, consistência, redundância e recuperação. Usar o caderno de perguntas como apoio.

Ensaio e limites do diagnóstico

O diagnóstico pode apontar redundância insuficiente em cada réplica A/B isolada. A redundância de leitura depende do conjunto e da seleção de destino saudável pela API. Explique o limite da heurística. Não use pontuação do painel como certificação de segurança, disponibilidade ou custo.

Ensaie com a versão instalada do Desktop, confira o resultado da importação da URL e os parâmetros dos Labs. Após alterações, aguarde **Salvo agora**. O código de referência usa dados sintéticos; o schema e o mapa descrevem controles que precisam de implementação e teste em um sistema real.

Cartão do apresentador

25 minutos · Build Arch Desktop · projeto final v9

Tempo	Tela/ação	Frase-chave
0-2	Arquitetura final	"Leitura e compra têm exigências diferentes."
2-6	Histórico > Trabalho atual	"Cada componente entrou para atender uma necessidade."
6-9	Checkout, banco, relay e DLQ	"Pedido e intenção de evento nascem na mesma transação."
9-15	Labs 01, 02 e 03	"Mais APIs continuam chegando ao mesmo banco."
15-18	Carga ao vivo 10x	"Fila crescente mostra a diferença entre demanda e capacidade."
18-22	Analisar projeto > GitHub público	"O código oferece evidências; o mapa precisa de revisão."
22-25	Falha e exportação	"Medir, testar falhas e registrar a decisão."

Parâmetros que devem permanecer iguais

Temporal: carga 2.000 req/min; Pico; 4x; 10 min virtuais; nenhuma falha. **Ao vivo - base:** 200 usuários × 10 ações/min; 30 s. **Ao vivo - pico:** 2.000 usuários × 10 ações/min; 30 s.

Esperado: Labs 01/02 = 895 ms e 68%; Lab 03 = 48 ms e 0,05% no temporal. Ao vivo no pico: aproximadamente 2.650 pendentes nos Labs 01/02; zero no Lab 03 após drenagem.

GitHub

<https://github.com/DevKaue/arquitetura-santa-cecilia> Referência **main**. Importar como outro projeto. Mostrar Compose > evidência > rascunho > revisão. Retornar ao projeto final antes do fechamento.

Conferência antes de começar

- Arquitetura final aberta, **Salvo agora**, canvas enquadrado.
- Três Labs conferidos; URL testada no Desktop; Internet disponível.
- Roteiro aberto e backups acessíveis para recuperação.
- Uma frase para declarar o limite: "Os números são saídas do modelo, não medições de produção."

Perguntas e respostas para discussão

Usar a **resposta breve** durante a palestra. O **aprofundamento** serve para perguntas adicionais, sem interromper o fluxo da demonstração.

A. Escolha e evolução da arquitetura

1. É necessário começar com microsserviços?

Resposta breve: não. Um monólito modular pode atender a fase inicial com menos custo operacional.

Aprofundamento: separar implantação faz sentido quando volume, autonomia da equipe, frequência de mudança ou isolamento de falhas justificam observabilidade, comunicação distribuída e operação adicionais. O mapa ensina responsabilidades; não impõe um estilo único.

2. Como decidir o que escalar?

Resposta breve: medir a dependência limitante antes de aumentar instâncias.

Aprofundamento: olhar saturação, filas, latência medida, erros, queries, índices, pool de conexões e concorrência. No Lab 02, seis APIs continuam chegando ao mesmo banco. O Lab 03 altera a capacidade cadastrada no limitador; a intervenção real precisa de medição.

3. Qual o custo dessa arquitetura?

Resposta breve: o painel apresenta valores didáticos; um orçamento real depende do uso e do provedor.

Aprofundamento: região, tráfego de saída, requests, processamento, armazenamento, observabilidade, suporte e operação precisam entrar na conta. Autoscaling exige teto e proteção das dependências compartilhadas.

B. Dados, cache e compra

4. O que significa o perfil 95/5?

Resposta breve: é a hipótese de negócio: 95% das chamadas leem e 5% escrevem.

Aprofundamento: no pico de 20.000 req/min, seriam 19.000 leituras e 1.000 escritas. Um HIT de 90% é uma segunda hipótese e deixa 1.900 leituras chegando ao banco. O motor de ramificações não aplica automaticamente esses percentuais.

5. Como impedir que dois clientes comprem a última unidade?

Resposta breve: decidir a reserva no writer com atualização condicional dentro da transação.

Aprofundamento: atualizar somente se houver estoque suficiente; fazer rollback de todos os itens se um falhar. Usar ordem estável para itens e tratar deadlock/retry com idempotência. Um teste concorrente deve provar que o saldo não fica negativo.

6. Cache ou réplica podem decidir a reserva?

Resposta breve: a tela pode aceitar atraso; a reserva usa o dado autoritativo.

Aprofundamento: TTL e replicação podem exibir estado antigo. A API de compra precisa conferir disponibilidade, preço e autorização no domínio apropriado. Falha do cache exige fallback limitado para não provocar avalanche no banco.

7. Standby aumenta a capacidade de escrita?

Resposta breve: não. O modelo mantém um writer e um standby para alta disponibilidade.

Aprofundamento: réplicas A/B atendem leituras do catálogo com atraso possível. HA, distribuição de leitura e capacidade de escrita são decisões distintas. A promoção deve impedir que o writer antigo continue aceitando escrita.

C. Resiliência e processamento assíncrono

8. Retry sempre melhora a disponibilidade?

Resposta breve: retry sem limite pode aumentar a sobrecarga e repetir efeitos.

Aprofundamento: timeout, backoff com jitter, orçamento finito e idempotência são necessários. Em pagamento, timeout significa resultado desconhecido: consultar estado ou reconciliar antes de compensar a reserva.

9. Outbox garante exatamente uma entrega?

Resposta breve: não. Ela grava pedido e intenção de publicar na mesma transação local.

Aprofundamento: o relay pode publicar, receber confirmação e cair antes de marcar o evento. A repetição exige deduplicação por evento/consumidor. O evento de compra confirmada nasce após reconciliação do pagamento; criar pedido não autoriza enviar confirmação de compra.

10. Como evitar e-mail duplicado?

Resposta breve: o efeito externo também precisa de uma política de idempotência.

Aprofundamento: gravar `processed_events` não torna atômica a chamada ao serviço de e-mail. Exigir idempotência do provedor ou documentar a tolerância a duplicata e a reconciliação. ACK deve acompanhar a política de efeito durável.

11. Para que serve a DLQ?

Resposta breve: retirar mensagens que esgotaram tentativas e permitir investigação e reprocessamento controlado.

Aprofundamento: monitorar idade/volume, definir responsável e retenção, corrigir a causa e repetir com deduplicação. DLQ sem processo operacional vira acúmulo de problema.

D. Segurança, operação e recuperação

12. WAF e token válido bastam para segurança?

Resposta breve: não. A aplicação também precisa validar entrada e autorizar cada recurso.

Aprofundamento: conferir o dono do pedido, menor privilégio no banco e nos serviços, rede privada para dados/broker, segredos por ambiente e logs sem tokens/cartão. WAF atua na borda; não conhece todas as regras do domínio.

13. Qual a disponibilidade real? E a recuperação de região?

Resposta breve: disponibilidade exige medição; RPO/RTO exigem teste de desastre.

Aprofundamento: definir SLI, período de observação e SLO. As metas propostas de $RPO \leq 5$ min e $RTO \leq 30$ min precisam de ensaio de restore, promoção, fencing e roteamento. CDN mundial não torna o writer active-active nem substitui backup.

E. Build Arch e repositório

14. Por que a arquitetura inferida é diferente do desenho final?

Resposta breve: os manifests revelam recursos e dependências; o desenho final incorpora decisões de negócio e operação.

Aprofundamento: a inferência inclui redes, volume, imagens e abstrações com possíveis sobreposições. Revisar evidências e separar inicialização de chamada real. Capacidade, segurança, consistência e fluxo de pagamento não são comprovados por `depends_on`.

15. A nota do diagnóstico e os números da simulação garantem o sistema?

Resposta breve: são indicadores do modelo, úteis para discutir hipóteses.

Aprofundamento: o diagnóstico examina blocos; a redundância A/B depende do roteamento saudável do conjunto. O simulador temporal e a carga ao vivo usam motores distintos. Um sistema real precisa de benchmark, teste concorrente, teste de falhas e observação.

16. O repositório implementa uma plataforma de compras completa?

Resposta breve: publica o exemplo arquitetural, manifests e endpoints demonstrativos.

Aprofundamento: o catálogo é sintético, checkout retorna 501 e relay/worker não processam eventos. O schema explicita invariantes; o comportamento de domínio está descrito nas notas técnicas. O objetivo ao importar do GitHub é revisar evidências de arquitetura.